Teaching LLMs to Solve Problems: Which Learning-Science Findings Carry Over?

Ying Chen

Working draft, November 2026 — results marked *preliminary* are being re-tested on fresh seeds.
github.com/yingchen-coding/game-solving-paper

Abstract

Practitioners borrow from pedagogy when they prompt or fine-tune language models: give worked examples, state the rules, encourage, practise more. We test, one finding at a time, whether the classic results from human learning hold for an LLM's problem-solving capability. We use small interactive problems with exact verifiers as both the practice field and the exam: four practice games, four extra practice games, two near-transfer problems and two far-transfer problems that are never trained on. On Qwen2.5-7B-Instruct we run 15 teaching conditions spanning prompts, in-the-moment teachers and LoRA fine-tuning. Four findings carry over from people: worked examples improve the practised task, how-to guidance helps where the model is a novice and hurts where it is already good (expertise reversal, preliminary), pressure hurts and encouragement does nothing, and practice does not transfer far. Two do not. The model rarely uses a free verifier it is given, and when it is fine-tuned on expert play that calls the verifier before every answer, it learns the habit—on never-practised problems CHECK calls rise from 2 of 99 actions to 70 of 114—and, with varied practice, learns to call an unseen tool correctly (well-formed calls 0% to 88%), yet solves no more problems and writes "all observations agree" for answers it never checked. Comparing two solved cases, which helps people transfer, does worse than stating the rules. What works is self-explanation: with the same states and expert moves, training on reasoning the model wrote itself instead of the expert's prose lifts the black-box task from 4/20 to 12/20 (the expert prose gives 0–2/20), and on-policy distillation agrees; both gains are on practised tasks. Fewer evidence-contradicting actions on never-practised problems come from having the verifier available, not from the training: the same distilled model contradicts its evidence in 13 of 20 episodes without the tool and 5 with it. Across all conditions one mechanism predicts failure: an episode containing an action that contradicts the model's own earlier evidence almost never succeeds.

1 Introduction

A model that is told "check your answer against the evidence" can repeat the instruction and still answer against the evidence. We wanted to know which kinds of teaching change what the model *does*, and whether any of it transfers to problems it never practised. Educational psychology has a century of results on exactly these questions for human learners, so we use them as hypotheses: each classic finding becomes one experimental condition, operationalised the same way on the model, measured on the same problems.

The comparison is useful in both directions. Where a finding carries over, practitioners can rely on it. Where it does not, the difference tells us something about how these models learn. Our clearest difference is that the model learns the *form* of verification—the tool call and the sentence

claiming agreement—without the computation, which connects problem-solving training to the problem of unfaithful reasoning [Turpin et al., 2023].

2 Setup

Problems. Every problem is an interactive text game with an exact verifier (Table 1). Each turn the model writes a short thought and one action. Practice games may be trained on; far-transfer problems are never trained on, used for prompt tuning, or used to write rules. Evaluation seeds start at 10000 and are disjoint from every training seed; a guard in the data builder refuses evaluation seeds.

Role	Problem	What it needs	Budget
Practice	GuessNumber	binary search from higher/lower feedback	8–9
	CodeBreak	deduce a 3-digit code from exact/misplaced counts	10
	HiddenRule	query a quadratic, then state its formula	6
	OddCoin	find the heavy coin with a balance	4–5
Extra practice	Heaviest, Radar, Boxes, Switches	comparisons, distance pings, yes/no elimination, combined tests	4–8
Near transfer	Lock, Machine	CodeBreak / HiddenRule skill, new words and verbs	—
Far transfer	Circuit	find the faulty stage of an 8-stage pipeline	5
	BugFix	fix two buggy functions, graded on hidden tests	14

Table 1: Problems. Every verifier is exact; nothing in the environment calls a model.

Model and decoding. Qwen2.5-7B-Instruct, 4-bit, MLX [Qwen Team, 2024]; greedy decoding; at most 256 tokens per turn. Fine-tuning uses LoRA [Hu et al., 2022] on 8 layers, 200 iterations, learning rate 2×10^{-5}, with gradient clipping. Each condition plays 20 problems per task.

Conditions. Prompts (a strategy prompt, a playbook the model writes from its own attempts in the style of Shinn et al. [2023], a playbook of expert rules), teachers that add one line after every turn (encourage, yell, remind, hint, name the contradiction), and LoRA fine-tuning on the model's own verified wins (expert iteration, Anthony et al., 2017, Zelikman et al., 2022) or on scripted expert play with its reasoning written out.

A free verifier. In some conditions every problem offers `CHECK <candidate>`, which reports whether a candidate is consistent with all evidence so far without revealing the answer and without using an action (at most 12 per problem).

Contradictions. For every action we compute, from the transcript alone, whether it contradicts earlier feedback (for example, answering a coin that an earlier weighing ruled out, or guessing a code that an earlier guess's feedback excludes).

Statistics. Fisher's exact test, two-sided; Holm correction over the ten comparisons chosen before looking at significance; bootstrap 95% intervals. Findings marked *preliminary* were noticed after looking at the data and are being re-tested on 40 fresh problems per task (seeds 10020–10059) with the analysis fixed in advance.

3 Results

3.1 Success by condition

Condition	Guess	Code	Rule	Coin	Circuit	BugFix	Practised	Never practised
Plain	20	0	4	16	3	0	40/80	3/40
Strategy prompt	20	0	10	11	4	3	41/80	7/40
Self-written playbook	20	3	5	16	1	0	44/80	1/40
Expert rules playbook	20	1	11	7	1	0	39/80	1/40
Fine-tune on own wins	20	1	10	16	1	1	47/80	2/40
Own wins + playbook	20	3	11	18	3	0	52/80	3/40
Fine-tune on worked examples	16	1	8	20	1	0	45/80	1/40
CHECK tool, untrained	20	1	4	9	0	0	34/80	0/40
Encourage	20	0	4	15	2	0	39/80	2/40
Yell	20	2	5	7	0	0	34/80	0/40
Remind of the task	20	4	4	14	3	0	42/80	3/40
Hint	20	1	2	16	2	1	39/80	3/40
Coach names contradictions	20	2	8	14	1	1	44/80	2/40
Check habit, 4 practice games	20	0	0	18	3	0	38/80	3/40
Check habit, 8 practice games	18	0	0	20	1	0	38/80	1/40
Analogical comparison prompt	20	0	3	11	0	1	34/80	1/40
Process-verified examples, 4 games	19	0	2	1	0	0	22/80	0/40
Process-verified examples, 8 games	16	1	2	15	1	0	34/80	1/40
Self-explanation, 8 games	20	0	12	18	1	0	50/80	1/40
On-policy distillation†	19	1	15	8	2	0	43/80	2/40
On-policy distillation + tool†	16	1	17	14	2	0	48/80	2/40
Model-written operational rules†	20	1	9	13	1	1	43/80	2/40
Distillation + operational rules†	19	0	10	8	2	1	37/80	3/40

Table 2: Problems solved out of 20 per task. "Check habit" conditions are fine-tuned on expert play that calls CHECK before every committing move and are evaluated with the tool available; the process-verified and self-explanation conditions are too. †Run by a second experimenter on the same machine and model.

None of the ten pre-chosen comparisons with the plain model is significant after Holm correction. The largest is fine-tuning on the model's own wins plus its playbook on the practised games, 52/80 vs 40/80 (+15 points, 95% CI $[0, +30]$, $p = .078$). On the never-practised problems no condition beats plain; the best is the strategy prompt, 7/40 vs 3/40 ($p = .31$).

3.2 The failure mechanism: acting against one's own evidence

Task	With a contradicting action	Without	p
OddCoin	21/88 (24%)	158/172 (92%)	2×10^{-29}
HiddenRule	0/156 (0%)	86/104 (83%)	3×10^{-51}
Circuit	0/162 (0%)	22/98 (22%)	9×10^{-11}
BugFix	2/253 (1%)	4/7 (57%)	3×10^{-6}

Table 3: Episode-level success, pooled over the first 13 conditions.

An episode with even one action that contradicts earlier feedback almost never succeeds (Table 3). Across conditions, a task's contradiction rate tracks its success (Spearman $\rho = -0.82$ on OddCoin, -0.78 on HiddenRule). Part of the association holds by construction where the contra-

dicting action is the final answer; the non-trivial part is that the evidence is in the model's context when it answers against it.

3.3 What carries over from people

Worked examples help the practised task [Sweller and Cooper, 1985]. Fine-tuning on expert play with its reasoning written out takes OddCoin from 16/20 to 20/20, with no answer contradicting the model's own weighings.

Expertise reversal [Kalyuga et al., 2003] (preliminary). Because any prompt change perturbs greedy outcomes, we compare guidance against the pooled light-touch conditions (encourage, remind, hint), which add text without telling the model how to work. On HiddenRule, where the model is a novice, the strategy prompt solves 10/20 ($p = .006$) and expert rules 11/20 ($p = .002$) against 10/60 for the controls. On OddCoin, where it is already good, expert rules solve 7/20 against 45/60 ($p = .002$). The harm runs through the mechanism above: with expert rules, 12 of 20 OddCoin answers name a coin the model's own weighings had ruled out (plain: 3).

Pressure hurts, encouragement does nothing [Yerkes and Dodson, 1908, Sisk et al., 2018] (preliminary). A teacher line that yells after every turn takes OddCoin from 16/20 to 7/20 ($p = .010$); 13 of 20 answers contradict the model's own weighings. Encouragement changes nothing on any task (39/80 vs 40/80 practised). The outcome matches people; the cause cannot be anxiety.

Far transfer is rare [Thorndike and Woodworth, 1901, Barnett and Ceci, 2002]. Every method that improves the practised games leaves the never-practised problems at or below the plain model.

3.4 What does not carry over: the check becomes a ritual

An untrained model given the free verifier rarely uses it (Circuit: 2 CHECK calls in 99 actions) and solves fewer problems (34/80 vs 40/80 practised). We therefore fine-tuned on expert play that calls CHECK on its candidate before every committing move, on the four practice games, and, to test variability of practice [Schmidt, 1975], on eight games with the same number of episodes (240) and examples (651).

	Untrained + tool	4 games	8 games
Circuit: CHECK calls / actions	2/99	70/114	61/124
BugFix: CHECK calls / actions	74/328	120/200	193/353
BugFix: well-formed calls	36/74	0/120	170/193
CodeBreak: commits a candidate the check rejected	18/29	72/135	12/54
Never-practised problems solved	0/40	3/40	1/40
HiddenRule solved (practised)	4/20	0/20	0/20

Table 4: Use of the free verifier (`check_use.py`).

The habit transfers: on never-practised problems the model now calls the verifier often (Table 4). Varied practice adds something real: on BugFix, whose tool takes a different argument, well-formed calls rise from 0% (four games) to 88% (eight games), and committing a candidate the check has just rejected falls from 53% to 22% on CodeBreak. None of it turns into solved problems. On

HiddenRule, a practised game, success falls from 4/20 to 0/20. The transcripts show why. The model writes, for example, "f(0) = −4, f(1) = −1, f(2) = 2 all agree with $x^2 - 5$" (the answer is $3x - 4$) and commits without calling the tool; on Circuit it calls `CHECK 36` with a value where a stage number is required and repeats the same call after the error. The validation loss of the eight-game run reached 0.0012: the model had memorised the sentences that accompany a check.

3.5 What works: training on the model's own reasoning

The process-verified condition writes every check's arithmetic into the expert's reasoning; the self-explanation condition keeps exactly the same states and expert moves but asks the base model to explain, in its own words, why the expert's move is right, and trains on that explanation instead. On-policy distillation, run independently, lets the model play under a structured verification prompt with a coach, keeps its successes and trains without the prompt. Both kinds of self-generated reasoning help where imitation of the expert's prose hurts (Table 2): on HiddenRule, self-explanation solves 12/20 against 4/20 for plain ($p = .02$) and 0–2/20 for the expert's prose ($p < .003$); distillation solves 15–17/20. With varied practice the process-verified model also learns to obey the verifier (committing a rejected candidate: 4 of 74 checks), but still cannot compute the right candidate.

	CHECK tool	Circuit	BugFix	Practised solved
Plain	no	13	20	40/80
Fine-tune on worked examples	no	12	18	45/80
On-policy distillation	no	13	20	43/80
Model-written operational rules	no	17	18	43/80
Distillation + operational rules	no	12	18	37/80
CHECK tool, untrained	yes	7	20	34/80
On-policy distillation + tool	yes	5	18	48/80
Check habit, 4 games	yes	2	20	38/80
Process-verified, 4 games	yes	0	13	22/80
Process-verified, 8 games	yes	4	19	34/80
Self-explanation, 8 games	yes	2	11	50/80

Table 5: Episodes (of 20) on the never-practised problems containing an action that contradicts the model's own earlier evidence, by whether the CHECK tool is available at evaluation.

Fewer contradictions on the never-practised problems come mainly from the CHECK tool, not from what the model was trained on (Table 5). On Circuit the five conditions without the tool have 12–17 contradicting episodes of 20 and the six with it have 0–7. Within one model the effect is causal: the distilled model contradicts its evidence in 13 Circuit episodes without the tool and 5 with it ($p = .025$). Self-explanation (2) is not distinguishable from distillation with the tool (5, $p = .41$), so we withdraw the earlier reading that self-explanation transfers the mechanism; it is the tool. What remains is the practised games: among conditions with the tool, the two trained on the model's own reasoning solve the most (self-explanation 50/80, distillation 48/80), and the three that imitate the expert's prose solve 22–38/80. BugFix shows no consistent pattern. Rules the model writes for itself after playing ("restate what is known, plug every candidate into every observed case, change one value at a time"), given as a prompt, do not help significantly (HiddenRule 9/20 vs 4/20, $p = .18$) and do not add to distillation (15 to 10/20, $p = .19$). These decompositions were made after seeing the data and are being re-tested on fresh seeds.

4 People versus LLMs

Finding in people	Prediction	This model	Same?
Worked examples	beat unguided practice	practised task 16→20/20	same
Expertise reversal	guidance helps novices, hurts experts	4→11/20 weak task; 16→7/20 strong task	same (prelim.)
Pressure (Yerkes–Dodson)	harms hard tasks	16→7/20	same outcome, other cause
Encouragement, mind-set	small effects at scale	none	same
Far transfer is rare	near only	none to far problems	same
Variability of practice	varied practice transfers better	better tool use, not more solved	partly
Given a free check	learners use it	rarely used	different
Imitating narrated checks	—	learns the claim, not the check	different
Analogical comparison [Gentner et al., 2003]	comparing cases transfers better than rules	worse than rules (3 vs 11/20 weak task)	different
Self-explanation [Chi et al., 1989]	own explanations transfer better	4→12/20 weak task, 50/80 practised; its gains on unpractised problems come from the tool	partly (practised only)
Self-generated rules	knowing a rule ≠ using it	model-written rules as prompt: 4→9/20, n.s.	same

Table 6: Each row is one classic finding, operationalised the same way on the model.

5 Limitations and next steps

One model family at one size; 20 problems per task detects only large effects (about ±35 points per task); greedy decoding makes any prompt change a perturbation, which is why guidance is compared with light-touch controls. Expertise reversal, pressure and self-explanation were found after looking at the data and are being re-tested on fresh seeds; near-transfer baselines on Lock and Machine are running. Next: learning from erroneous examples, interleaved versus blocked practice, combining self-explanation with on-policy practice, and a second model family.

6 Related work

Expert iteration and self-taught reasoning fine-tune on a model's own verified successes [Anthony et al., 2017, Zelikman et al., 2022]; verbal reflection stores lessons in context [Shinn et al., 2023]. Generalisation from procedurally generated training levels improves with their number in reinforcement learning [Cobbe et al., 2019]. Chain-of-thought explanations can misrepresent what drives a model's answer [Turpin et al., 2023]; our check-habit result is a training-induced instance of the same gap.

References

Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems*, 2017.

Susan M. Barnett and Stephen J. Ceci. When and where do we apply what we learn? a taxonomy for far transfer. *Psychological Bulletin*, 128(4):612–637, 2002.

Michelene T. H. Chi, Miriam Bassok, Matthew W. Lewis, Peter Reimann, and Robert Glaser. Self-explanations: How students study and use examples in learning to solve problems. *Cognitive Science*, 13(2):145–182, 1989.

Karl Cobbe, Oleg Klimov, Chris Hesse, Taehoon Kim, and John Schulman. Quantifying generalization in reinforcement learning. In *International Conference on Machine Learning*, 2019.

Dedre Gentner, Jeffrey Loewenstein, and Leigh Thompson. Learning and transfer: A general role for analogical encoding. *Journal of Educational Psychology*, 95(2):393–408, 2003.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, et al. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations*, 2022.

Slava Kalyuga, Paul Ayres, Paul Chandler, and John Sweller. The expertise reversal effect. *Educational Psychologist*, 38(1):23–31, 2003.

Qwen Team. Qwen2.5 technical report. arXiv:2412.15115, 2024.

Richard A. Schmidt. A schema theory of discrete motor skill learning. *Psychological Review*, 82(4): 225–260, 1975.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.

Victoria F. Sisk, Alexander P. Burgoyne, Jingze Sun, Jennifer L. Butler, and Brooke N. Macnamara. To what extent and under which circumstances are growth mind-sets important to academic achievement? two meta-analyses. *Psychological Science*, 29(4):549–571, 2018.

John Sweller and Graham A. Cooper. The use of worked examples as a substitute for problem solving in learning algebra. *Cognition and Instruction*, 2(1):59–89, 1985.

Edward L. Thorndike and Robert S. Woodworth. The influence of improvement in one mental function upon the efficiency of other functions. *Psychological Review*, 8(3):247–261, 1901.

Miles Turpin, Julian Michael, Ethan Perez, and Samuel R. Bowman. Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting. In *Advances in Neural Information Processing Systems*, 2023.

Robert M. Yerkes and John D. Dodson. The relation of strength of stimulus to rapidity of habit-formation. *Journal of Comparative Neurology and Psychology*, 18(5):459–482, 1908.

Eric Zelikman, Yuhuai Wu, Jesse Mu, and Noah D. Goodman. STaR: Bootstrapping reasoning with reasoning. In *Advances in Neural Information Processing Systems*, 2022.